Universidad Francisco de Vitoria

SecureBox

Criptografía

Javier Vázquez Pereda

Lara de la O Torres
Rodrigo Justo Pérez

Abril 2026

1. Introducción

La protección de la información constituye una necesidad fundamental en los sistemas digitales actuales. El intercambio de documentos, credenciales, mensajes y datos sensibles a través de redes públicas exige mecanismos capaces de impedir accesos no autorizados, detectar modificaciones no legítimas y verificar la identidad de los participantes en la comunicación.

Tal y como se expone en la asignatura, la criptografía puede entenderse como el conjunto de técnicas y métodos destinados a transformar la información con el objetivo de proporcionar confidencialidad, integridad y autenticidad. Estos principios representan la base de numerosos sistemas actuales como HTTPS, SSH o VPN, utilizados diariamente tanto en Internet como en entornos corporativos y profesionales.

En este contexto se desarrolla SecureBox, una aplicación implementada en Python orientada a la protección segura de archivos mediante técnicas criptográficas modernas. El proyecto combina algoritmos simétricos y asimétricos siguiendo un modelo de cifrado híbrido, incorpora firmas digitales para garantizar la autoría e integridad de los datos, y añade una simulación de establecimiento de canal seguro entre dos entidades.

El sistema ha sido diseñado mediante una interfaz de línea de comandos (CLI), permitiendo generar claves criptográficas, cifrar archivos, descifrarlos, firmarlos digitalmente, verificar firmas e inspeccionar contenedores seguros creados por la aplicación. Esta aproximación facilita la automatización de pruebas, la reproducibilidad de resultados y el enfoque técnico de la práctica.

De esta forma, SecureBox permite aplicar de manera práctica numerosos conceptos estudiados en la asignatura, trasladando la teoría criptográfica a una implementación funcional y realista basada en tecnologías y mecanismos presentes en sistemas actuales.

2. Diseño general del sistema

SecureBox ha sido diseñado siguiendo una arquitectura modular, separando las distintas funciones criptográficas en componentes independientes. Esta organización facilita el mantenimiento del código, la reutilización de funciones y futuras ampliaciones del sistema.

La aplicación se ejecuta mediante una interfaz de línea de comandos (CLI), desde la cual el usuario puede invocar las distintas operaciones disponibles. Internamente, cada comando utiliza los módulos responsables de la gestión de claves, cifrado, firma digital o canal seguro.

El diseño sigue una filosofía habitual en sistemas reales, donde distintos mecanismos cooperan entre sí para ofrecer seguridad completa: cifrado simétrico para eficiencia, criptografía asimétrica para intercambio de claves y firmas digitales para autenticación.

2.1 Arquitectura general

Los módulos principales del sistema son:

- cli.py: interfaz principal del programa.
- keys.py: generación y gestión de claves.
- hybrid.py: cifrado híbrido RSA y ECC.
- aead.py: cifrado AES-GCM.
- signatures.py: firma y verificación Ed25519.
- formats.py: contenedor .sbox.
- handshake.py: canal seguro entre usuarios.

2.2 Flujo de funcionamiento

El funcionamiento general consiste en:

1. Generar claves criptográficas.
2. Cifrar archivos mediante AES-GCM y proteger la clave con RSA o ECC.
3. Firmar digitalmente el contenedor si se desea.
4. Verificar la firma con la clave pública.
5. Descifrar el contenido original con la clave privada.

2.3 Criterios de diseño aplicados

Durante el desarrollo se siguieron los siguientes criterios:

- Modularidad
- Seguridad
- Extensibilidad
- Legibilidad
- Usabilidad técnica

3. Gestión de claves

La gestión de claves constituye uno de los elementos más importantes de cualquier sistema criptográfico, ya que la seguridad no depende únicamente de los algoritmos utilizados, sino también de cómo se generan, almacenan y protegen las claves asociadas.

En SecureBox se emplean distintos tipos de claves según la funcionalidad requerida, evitando reutilizaciones innecesarias y siguiendo buenas prácticas habituales.

3.1 Tipos de claves

El sistema utiliza tres tipos principales de claves criptográficas:

- RSA: cifrado asimétrico y encapsulado de claves simétricas.

- X25519: intercambio seguro de secretos compartidos mediante curva elíptica.
- Ed25519: firma digital y verificación de autenticidad.

La utilización diferenciada de estos algoritmos permite aprovechar las ventajas específicas de cada uno.

3.2 Generación de claves

Las claves se generan mediante comandos propios del sistema, utilizando aleatoriedad criptográficamente segura para garantizar valores robustos e impredecibles.

Cada par generado está compuesto por una clave privada y una clave pública asociada.

python -m securebox.cli keygen --password 1234

3.3 Almacenamiento y formato PEM

Las claves se almacenan en formato **PEM**, uno de los estándares más utilizados en aplicaciones criptográficas reales. Este formato facilita la compatibilidad con otras herramientas y permite representar claves mediante texto estructurado.

3.4 Protección de claves privadas

Las claves privadas constituyen el elemento más sensible del sistema, por lo que se almacenan protegidas mediante contraseña. De esta forma, aunque un tercero accediera al archivo correspondiente, no podría utilizar la clave sin conocer la contraseña asociada.

4. Diseño del contenedor .sbox

Con el objetivo de almacenar de forma estructurada toda la información necesaria durante los procesos de cifrado y firma, en SecureBox se diseñó un contenedor propio denominado .sbox. Este archivo reúne tanto los datos cifrados como los metadatos criptográficos necesarios para su posterior verificación y recuperación.

```
● (.venv) PS C:\Users\Lara\CRIPTOGRAFIA> python -m securebox.cli encrypt mensaje.txt salida.sbox --key rsa_public.pem
  Archivo cifrado
○ (.venv) PS C:\Users\Lara\CRIPTOGRAFIA>
```

El uso de un formato específico permite centralizar en un único archivo todos los elementos generados durante la operación criptográfica, evitando gestionar múltiples archivos independientes.

4.1 Estructura del formato

El contenedor .sbox se basa en una estructura tipo **JSON**, elegida por su simplicidad, legibilidad y facilidad de procesamiento desde Python.

salida.sbox


```
{
  "version": "sbox-1",
  "mode": "rsa",
  "enc_alg": "aes_256_gcm",
  "wrap_alg": "rsa_oaep_sha256",
  "recipient_id": "94f9a81721a1d8ba1baedac522a6abdc6f4ddd145d71c3185a9d504f20373a6d",
  "nonce": "bP5WhzCrX5VqVyb1",
  "ciphertext": "1FLGX1DFb5bKaMSdFjnBAW4pLv6zTnvRMI8IHMxqATq/PH4UnFmC0iocxw2WQ/IJc16R0tgK82k=",
  "wrapped_key": "SGkDUhzTNqMjS7486dEniRNMttfBhL6amm/Af15F3yILK3Ob9aVyM1GMBU6HjAV3wIpY7eq0Lf50BZsHvn9
  "ephemeral_public_key": null,
  "salt": null,
  "signature": null
}
```


Cada archivo contiene pares clave-valor con la información necesaria para interpretar correctamente el contenido protegido, incluyendo:

- versión del formato
- modo criptográfico utilizado
- algoritmos empleados
- datos cifrados
- material criptográfico auxiliar
- firma digital

4.2 Campos del contenedor

Los principales campos implementados son:

- **version**: versión del formato.
- **mode**: modo RSA o ECC.
- **enc_alg**: algoritmo de cifrado simétrico.
- **wrap_alg**: mecanismo de protección de clave.
- **nonce**: valor aleatorio usado en AES-GCM.
- **ciphertext**: contenido cifrado.
- **wrapped_key**: clave simétrica protegida (RSA).
- **ephemeral_public_key**: clave pública efímera (ECC).
- **signature**: firma digital del contenedor.
- **sig_alg**: algoritmo de firma utilizado.


```
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA> python -m securebox.cli inspect salida.sbox
version: sbox-1
mode: rsa
enc_alg: aes_256_gcm
wrap_alg: rsa_oaep_sha256
recipient_id: 94f9a81721a1d8ba1baedac522a6abdc6f4ddd145d71c3185a9d504f20373a6d
nonce: bP5WhzCrX5VqVyb1
ciphertext: 1FLGX1DFb5bKaMSdFjnBAW4pLv6zTnvRMI8IHMxqATq/PH4UnFmC0iocxw2WQ/IJc16R0tgK82k=
wrapped_key: SGkDUhzTNqMjS7486dEniRNMttfBhL6amm/Af15F3yILK3Ob9aVyM1GMBU6HjAV3wIpY7eq0Lf50BZsHvn9qnlbO7Z2gTDkeW/vlro6grd
U0xQMVVD379OytFy29k3xjZz0DDU7ee70QeXkFnFxzn6RH0mXp8W5Z2/EbmfKJhJmHJZG2tPT8bEsLVsOyMwDJR6MmzQMCs9Gv7jTEGF7hqhQ19ylIVmf6i
ydbmJZdu6Vnq0eDej13+ojVUlpun/b6VzBo8Gi73T6OXQwV6iqHzYcy4CC2JBF18OMKAYnLFB9UBLMMl8/o+DpMn4B3jKv2ucVWQTvMfM7C8WB31hnZvw==
ephemeral_public_key: None
salt: None
signature: Nag3EArk1PiazQ7J36WvhX2XXt5BM/qMHJ9tbEYRpb8e4K3hF5eKBkcUP2BXGUjBk+uvbSqu9S7/wPzEkMX3Ag==
sig_alg: ed25519
```


No todos los campos aparecen simultáneamente, ya que algunos dependen del modo de cifrado seleccionado.

4.3 Codificación Base64

Diversos elementos del contenedor contienen datos binarios no representables directamente en JSON. Para resolverlo, se utiliza codificación **Base64**, transformando dichos datos en texto seguro para su almacenamiento.

Gracias a esta decisión, SecureBox puede almacenar información criptográfica compleja en un archivo legible y fácilmente procesable.

Los datos binarios se codifican en Base64 para permitir su almacenamiento textual.

5. Cifrado híbrido

Uno de los mecanismos principales implementados en SecureBox es el cifrado híbrido, modelo ampliamente utilizado en sistemas criptográficos reales por combinar las ventajas de la criptografía simétrica y la criptografía asimétrica.

Los algoritmos simétricos destacan por su velocidad y eficiencia, lo que los convierte en una opción adecuada para cifrar archivos completos o grandes volúmenes de información. Sin embargo, presentan la dificultad de intercambiar la clave de forma segura entre emisor y receptor. Por otro lado, los algoritmos asimétricos permiten resolver este problema mediante el uso de claves públicas y privadas, aunque resultan menos eficientes para cifrar grandes cantidades de datos.

Para aprovechar los beneficios de ambos enfoques, SecureBox cifra el contenido real del archivo mediante AES-GCM, mientras que la clave simétrica generada para dicha operación se protege posteriormente mediante mecanismos de clave pública.

De esta forma se obtiene un sistema seguro, eficiente y escalable.

```
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA> python -m securebox.cli encrypt mensaje.txt salida.sbox --key rsa_public.pem
Archivo cifrado
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA>
```

5.1 Modo RSA

En este modo, la clave simétrica utilizada para cifrar el archivo se protege mediante criptografía RSA utilizando la clave pública del destinatario.

El proceso seguido es el siguiente:

- Generación de una clave simétrica temporal aleatoria.
- Cifrado del contenido del archivo mediante AES-GCM.
- Protección de la clave simétrica mediante RSA-OAEP.
- Almacenamiento de los datos en el contenedor .sbox.

Durante el descifrado, únicamente el poseedor de la clave privada RSA correspondiente podrá recuperar la clave simétrica y acceder al contenido original.

5.2 Modo ECC

El segundo modo implementado utiliza criptografía de curva elíptica, ofreciendo una alternativa moderna y eficiente al uso de RSA.

En este caso, emisor y receptor generan un secreto compartido mediante intercambio de claves **X25519**. A partir de dicho secreto, el sistema deriva una clave simétrica válida utilizando **HKDF**, que posteriormente será empleada para cifrar la información mediante AES-GCM.

El proceso general es:

- Generación de clave efímera del emisor.
- Cálculo del secreto compartido mediante X25519.
- Derivación de clave mediante HKDF.
- Cifrado del archivo con AES-GCM.
- Inclusión de la clave pública efímera en el contenedor .sbox.

Este modo proporciona claves más pequeñas, buen rendimiento y un alto nivel de seguridad, siendo similar al enfoque utilizado en numerosos protocolos actuales.

6. Firmas digitales

Para reforzar la autenticidad e integridad de los archivos protegidos, se implementó un sistema de firmas digitales mediante Ed25519. Una vez generado el contenedor .sbox, el emisor calcula una firma usando su clave privada, y posteriormente cualquier receptor puede verificarla con la clave pública correspondiente. Este mecanismo permite garantizar la autoría del archivo, asegurar que el contenido no ha sido modificado y detectar cualquier alteración realizada sobre el contenedor. Si se cambia cualquiera de los campos firmados, la verificación falla automáticamente.

```
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA> python -m securebox.cli sign salida.sbox --key sign_private.pem --password 1234
Archivo firmado
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA>
```

```
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA> python -m securebox.cli verify salida.sbox --key sign_public.pem
Firma válida: True
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA>
```

```
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA> python .\script_test_signatures.py
SBOX firmado: {'version': 'sbox-1', 'mode': 'rsa', 'enc_alg': 'aes_256_gcm', 'wrap_alg': 'rsa_oaep_sha256', 'recipient_
id': 'dummy', 'nonce': 'datRZsXO0GTxkmhi', 'ciphertext': 'O0Bg+tW/4+/w1f+dN2l0DjVbNu172uOmvuC6gccfuWiMl4IP6dtW', 'wrapp
ed_key': 'hzsbBYGSjsH3CjXUijXePS5vIeLPfLwjko78gBTKtf1nAiPHX1b7sfVM2q4ypwPeRerREv22FBMowRaW/nSlTGsonH2yS8iMD8vW09NLe7AVB
1gKWVu/lHR/uy99cNBBNXQCePzV1/vbFtIFXKSQmTXHMcafLJMoHeG5RdZSeIX/dospO1f2MnskxNMh2LJ321YApAkMpHflOAC8S+mjZDPRGk1NeQcqKI0U
IGFWNI3x+StBwoDvwqS6cUdS0exOtlxkOf38DzufHrGbD/A/pb8SKOSMLhJC8oTttNsHoaF/YVgG6FMvHauvd6ALUu22OlfIOEix5t9SuSpmjFkpMA==',
'ephemeral_public_key': None, 'salt': None, 'signature': '2xBKK1X0zvZr6zFPpupI8ne50gClgAY8aU0xvzUMbJwEzHA+53k8ljbgMxwxj
DY48tn5C1RInvOzTROIaz0nBQ==', 'sig_alg': 'ed25519'}
VERIFICACION: True
VERIFICACION tras manipulación: False
```

7. Handshake seguro

Se implementó una simulación de handshake seguro entre dos usuarios, Alice y Bob, inspirada en protocolos como TLS. Su objetivo es demostrar cómo establecer un canal confidencial y autenticado sin compartir previamente una clave simétrica.

El proceso comienza con la generación de claves efímeras X25519 por ambas partes, intercambiando únicamente sus claves públicas. A partir de ellas, cada usuario calcula localmente un secreto compartido, evitando su transmisión directa.

Este secreto se procesa mediante HKDF-SHA256 para derivar las claves de sesión, generando claves separadas para envío y recepción. Además, se incluye contexto criptográfico para vincular las claves al propio proceso de establecimiento.

Para garantizar la autenticidad, se construye un transcript del intercambio, que se firma con Ed25519, permitiendo detectar alteraciones durante el handshake.

Una vez establecido el canal, los mensajes se cifran con AES-GCM, asegurando confidencialidad e integridad. El uso de un contador permite detectar modificaciones y ataques de repetición simples.

En conjunto, este módulo reproduce de forma simplificada los principios de los protocolos modernos: intercambio efímero de claves, derivación segura, autenticación del proceso y cifrado protegido de la comunicación.

```
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA> python -m securebox.cli handshake-demo
=== HANDSHAKE DEMO SECUREBOX ===
1. Claves efímeras X25519 generadas
2. Secreto compartido calculado correctamente
3. Claves de sesión derivadas con HKDF
4. Transcript autenticado correctamente
5. Intercambio de mensajes AEAD:
   alice -> bob: b'Hola Bob, soy Alice'
   bob -> alice: b'Hola Alice, recibido'
   alice -> bob: b'Te envio el mensaje 3'
   bob -> alice: b'Recibido el mensaje 4'
   alice -> bob: b'Ultimo mensaje cifrado'
=== HANDSHAKE COMPLETADO OK ===
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA>
```

8. Interfaz CLI

El sistema se ha diseñado para ser utilizado mediante una interfaz de línea de comandos (CLI), lo que permite ejecutar de forma sencilla y reproducible todas las funciones principales de SecureBox. A través de esta interfaz es posible generar claves, cifrar y descifrar archivos, firmar y verificar contenedores .sbox, inspeccionar su contenido y ejecutar la demostración del handshake seguro.

El uso de una CLI facilita la automatización de pruebas, la repetición de experimentos y la validación práctica del sistema, además de ofrecer una forma clara de interactuar con cada una de las funcionalidades implementadas. De esta manera, la herramienta puede

utilizarse de extremo a extremo sin necesidad de interfaz gráfica, manteniendo un funcionamiento directo y transparente.

Comandos principales:

```
1. python -m securebox.cli keygen --password 1234

2. python -m securebox.cli encrypt mensaje.txt salida.sbox --key rsa_public.pem

3. python -m securebox.cli sign salida.sbox --key sign_private.pem --password 1234

4. python -m securebox.cli verify salida.sbox --key sign_public.pem

5. python -m securebox.cli decrypt salida.sbox resultado.txt --key rsa_private.pem --
password 1234

6. python -m securebox.cli inspect salida.sbox

7. python -m securebox.cli handshake-demo
```

Esta interfaz facilita pruebas reproducibles y automatización.

9. Pruebas

Se realizaron pruebas automáticas mediante pytest y validaciones manuales desde CLI.

Prueba	Resultado
RSA encrypt/decrypt	Correcto
ECC encrypt/decrypt	Correcto
Sign/verify	Correcto
Tampering detectado	Correcto
Handshake demo	Correcto
Flujo CLI completo	Correcto

Se ha implementado 25 pruebas automáticas mediante pytest, con el objetivo de validar tanto el funcionamiento correcto del sistema como la detección de errores y manipulaciones. En concreto, se han desarrollado 15 pruebas unitarias, centradas en la gestión de claves, la derivación de claves mediante HKDF, el formato del contenedor .sbox, la codificación en Base64 y el cifrado simétrico con AES-256-GCM.

Además, se han incluido 10 pruebas de integración para comprobar el funcionamiento conjunto de los distintos módulos del sistema. Estas pruebas validan los flujos completos de cifrado -> descifrado en modo RSA y ECC, el proceso de firma -> verificación, el guardado y carga de contenedores .sbox, y el establecimiento de un canal seguro mediante el módulo de handshake, incluyendo el intercambio de mensajes cifrados.

Junto a ello, se han implementado 8 pruebas negativas, destinadas a comprobar que el sistema detecta correctamente situaciones de error o manipulación. Entre ellas se

incluyen casos de ciphertext modificado, tag incorrecto, firma incorrecta, contraseña incorrecta, alteración del contenido firmado y detección de replay en el handshake. En conjunto, esta batería de pruebas permite verificar tanto la funcionalidad principal de SecureBox como su comportamiento ante escenarios adversos.

salida.sbox


```
{
  "version": "sbox-1",
  "mode": "rsa",
  "enc_alg": "aes_256_gcm",
  "wrap_alg": "rsa_oaep_sha256",
  "recipient_id": "94f9a81721a1d8ba1baedac522a6abdc6f4ddd145d71c3185a9d504f20373a6d",
  "nonce": "bP5WhzCrX5VqVyb1",
  "ciphertext": "1FLGX1DFb5bKaMSdFjnBAW4pLv6zTnvRMI8IHMxqATq/PH4UnFmC0iocxw2WQ/IJc16R0tgK82k=",
  "wrapped_key": "SGkDUhzTNqMjS7486dEniRNMttfBhL6amm/Af15F3yILK3Ob9aVyM1GMBU6HjAV3wIpY7eq0Lf50BZsHvn9
  "ephemeral_public_key": null,
  "salt": null,
  "signature": null
}
```


```
(.venv) PS C:\Users\Lara\CRIPTOGRAFIA> pytest -v
========================================== test session starts ==========================================
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Lara\CRIPTOGRAFIA\.venv\Scripts\python.ex
e
cachedir: .pytest_cache
rootdir: C:\Users\Lara\CRIPTOGRAFIA
collected 25 items

tests/test_aead.py::test_aead_encrypt_decrypt_roundtrip PASSED                                  [  4%]
tests/test_aead.py::test_modified_ciphertext_is_rejected PASSED                                  [  8%]
tests/test_aead.py::test_modified_tag_is_rejected PASSED                                         [ 12%]
tests/test_formats.py::test_base64_roundtrip PASSED                                              [ 16%]
tests/test_formats.py::test_save_and_load_sbox PASSED                                            [ 20%]
tests/test_formats.py::test_canonicalize_excludes_signature_fields PASSED                        [ 24%]
tests/test_formats.py::test_canonicalize_is_stable_for_equivalent_payloads PASSED                [ 28%]
tests/test_handshake.py::test_handshake_transcript_authenticates_peers PASSED                    [ 32%]
tests/test_handshake.py::test_handshake_exchanges_five_encrypted_messages PASSED                 [ 36%]
tests/test_handshake.py::test_handshake_replay_is_rejected PASSED                                [ 40%]
tests/test_handshake.py::test_handshake_tampered_ciphertext_is_rejected PASSED                   [ 44%]
tests/test_hybrid.py::test_rsa_encrypt_decrypt PASSED                                            [ 48%]
tests/test_hybrid.py::test_ecc_encrypt_decrypt PASSED                                            [ 52%]
tests/test_hybrid.py::test_rsa_encrypt_save_load_decrypt PASSED                                  [ 56%]
tests/test_kdf.py::test_hkdf_same_inputs_produce_same_key PASSED                                 [ 60%]
tests/test_kdf.py::test_hkdf_different_salt_changes_key PASSED                                   [ 64%]
tests/test_kdf.py::test_hkdf_respects_requested_length PASSED                                    [ 68%]
tests/test_keys.py::test_rsa_key_serialization_roundtrip PASSED                                  [ 72%]
tests/test_keys.py::test_ecdh_key_serialization_roundtrip PASSED                                 [ 76%]
tests/test_keys.py::test_sign_key_serialization_roundtrip PASSED                                 [ 80%]
tests/test_keys.py::test_invalid_rsa_key_size_raises_value_error PASSED                          [ 84%]
tests/test_keys.py::test_private_key_wrong_password_raises_value_error PASSED                    [ 88%]
tests/test_signatures.py::test_sign_and_verify PASSED                                            [ 92%]
tests/test_signatures.py::test_signature_tampering PASSED                                        [ 96%]
tests/test_signatures.py::test_signature_invalid_with_wrong_public_key PASSED                    [100%]

========================================== 25 passed in 0.44s ==========================================
```

10. Rendimiento, dificultades y conclusiones

Rendimiento

Las pruebas realizadas sobre archivos de ejemplo muestran que el sistema presenta un rendimiento adecuado para el tipo de operaciones implementadas. La generación de claves, así como los procesos de cifrado, descifrado, firma y verificación, se ejecutan en tiempos reducidos, generalmente inferiores a un segundo en condiciones normales. Esto se debe principalmente al uso de algoritmos eficientes como AES-GCM para el cifrado de

datos y a que las operaciones asimétricas se limitan a la protección de claves o a la firma de información. En conjunto, el comportamiento del sistema es fluido y permite su uso práctico sin penalizaciones apreciables en tiempo de ejecución.

Dificultades encontradas

Durante el desarrollo del proyecto surgieron diversas dificultades técnicas que condicionaron parcialmente la implementación. Una de las principales fue la correcta separación y uso de distintos tipos de claves, especialmente entre RSA (cifrado) y Ed25519 (firma), lo que obligó a establecer una organización clara en el sistema.

Otra complicación relevante fue la serialización del contenedor .sbox, especialmente en lo relativo a la firma, ya que fue necesario asegurar que los datos se mantuvieran consistentes para poder verificar correctamente su integridad.

Asimismo, la integración del módulo de handshake dentro de la interfaz CLI requirió ajustes adicionales para mantener coherencia con el resto de comandos. Por último, la validación de firmas tras modificaciones en el contenedor también presentó ciertos retos, ya que fue necesario garantizar que cualquier alteración fuera detectada de forma fiable.

Conclusión

El desarrollo de SecureBox ha permitido aplicar de forma práctica los conceptos criptográficos estudiados en la asignatura, integrándolos en una solución completa y funcional. A través de la combinación de cifrado híbrido, firmas digitales y establecimiento de canal seguro, se ha conseguido construir un sistema que refleja, a pequeña escala, el funcionamiento de mecanismos utilizados en entornos reales.

En conjunto, el proyecto cumple los objetivos planteados y demuestra una comprensión sólida de los fundamentos de la criptografía aplicada, así como su implementación en un contexto práctico.